Dirty Access Tracking in GPU-UVM

Aditi Khandelia, Arush Upadhyaya, Kushagra Srivastava, Sankalp Mittal, Vidhi Jain

CS614 Course Project
Indian Institute of Technology, Kanpur

March 21, 2026

Table of Contents

Introduction

- NVIDIA's **Unified Virtual Memory (UVM)** framework presents a shared address space to the CPU and GPU, servicing page residency via a hardware fault-driven migration mechanism
- The existing UVM implementation provides no facility to track *which* virtual pages have been written by the GPU, nor the temporal ordering of such writes
- This work presents a **dirty page tracking** subsystem integrated directly into the UVM kernel module, offering:
  - Instrumentation of the GPU write-fault servicing path to record per-page first-write timestamps
  - Per-process isolation of tracking state via nested `xarray` structures indexed by PID
  - A `procfs`-based query interface supporting address range filtering
- The system enables precise post-hoc attribution of GPU memory writes

Motivation

- **Fault-tolerant training:** Large ML training jobs that crash must restart from scratch; knowing which model parameter pages were updated enables incremental checkpointing and reduces recovery time
- **GPU process live migration:** Migrating a running GPU process between nodes requires identifying dirty pages to minimise data transfer; no such mechanism exists in the UVM stack today
- **Memory overcommitment:** Operating systems swap CPU pages using dirty bits; an analogous mechanism for GPU memory requires kernel-level write tracking that hardware does not expose directly
- **Heterogeneous synchronisation:** In CPU-GPU workloads with alternating phases, only pages written by the GPU need to be synchronised back to the host

Implementation Idea

- GPU write faults are already handled by the UVM kernel module to migrate pages; this fault path is the natural point at which a write can be observed
- On each write fault, record the faulting page number and a kernel timestamp into a per-process data structure; subsequent writes to the same page are ignored
- Maintain one table per process, so that tracking for one process does not interfere with another
- Expose the accumulated dirty set to userspace via procfs, allowing any tool to query which pages were written and when without modifying the application
- Provide an explicit start/stop interface so that tracking is scoped to a region of interest, and a fresh table is guaranteed on each start

Implementation: Data Structures

- Two nested `xarray` structures form the core of the tracking state:
  - A global `xarray` maps each PID to a `uvm_dirty_page_table` instance
  - Each `uvm_dirty_page_table` contains a second `xarray` mapping page numbers to `dirty_page_info` entries
- Each `dirty_page_info` record stores the page number, a `ktime_get_ns()` timestamp captured at fault-service time (not at the write itself), and the PID
- `xarray` is used throughout for its lock-free read semantics and O(1) indexed lookup; allocations in the fault path use `GFP_ATOMIC` to avoid sleeping in interrupt context
- First-write-wins semantics: if an entry already exists for a page, the record call is a no-op, preserving the earliest observed timestamp

Implementation: Fault Hook

- **Invalidation on start:** When tracking is enabled, all existing GPU page table mappings for the process are invalidated, forcing fresh faults on subsequent accesses so that no write goes unobserved.
- **On a write fault:** A conditional is inserted into the GPU replayable fault handler in `uvm_gpu_replayable_faults.c`; if tracking is active for the faulting PID, the page address and timestamp are recorded in the dirty table before the mapping is restored and the instruction is replayed.
- **On a read fault:** A conditional is inserted into the VA block fault-servicing path in `uvm_va_block.c`; if tracking is active, the page is re-mapped read-only instead of read-write, ensuring that any subsequent write to that page still generates a fault and is recorded rather than completing silently through an already-valid mapping.

Implementation: procfs Interface

Five entries are exposed under `/proc/driver/nvidia-uvm/`:

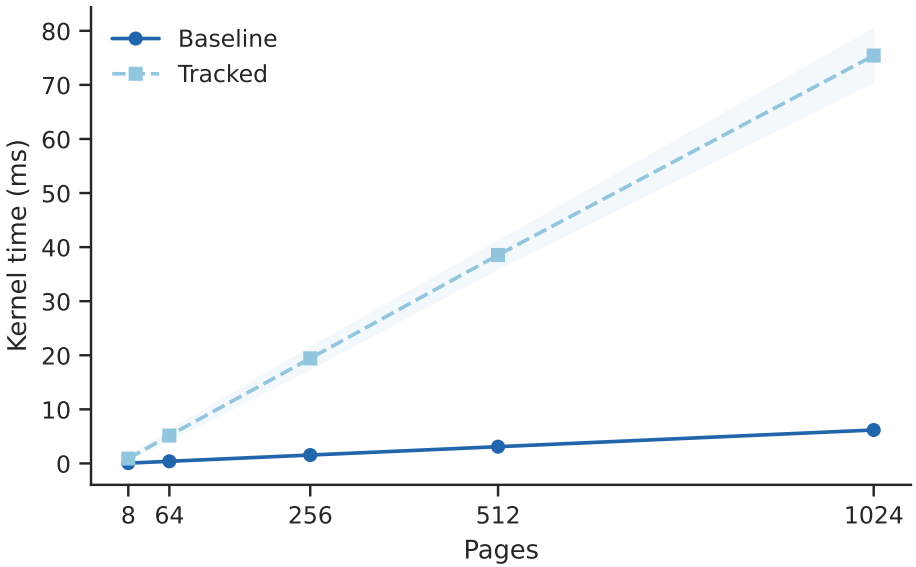
- `dirty_pids_start_track`: initialise a fresh dirty page table for the calling process and invalidate existing GPU mappings
- `dirty_pids_stop_track`: destroy the table for the calling process and cease recording
- `dirty_pid_to_query`: select which process's table is read on the next `dirty_pages` access
- `dirty_range`: set a virtual address range filter; only pages within the range are returned
- `dirty_pages`: read out recorded entries as `(address, timestamp, pid)` tuples; reads are non-destructive

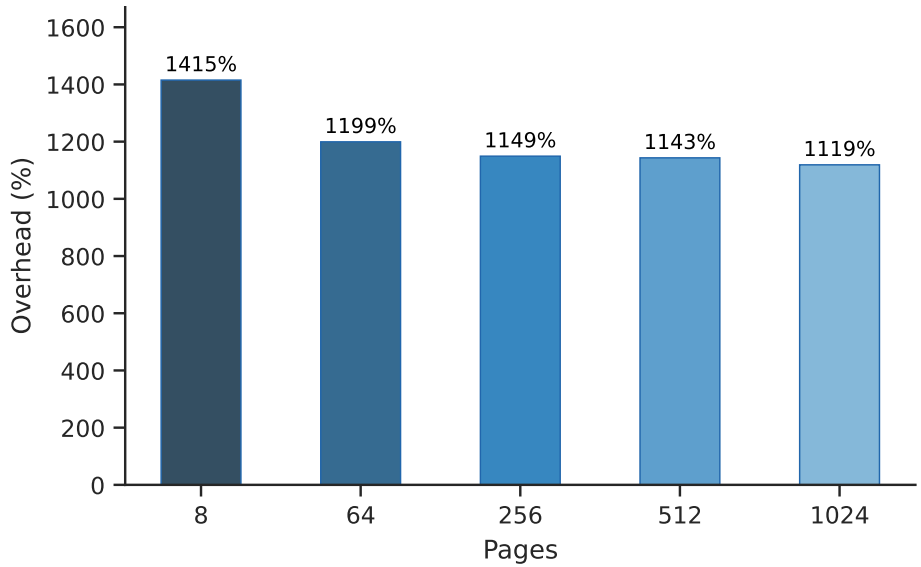
Testing Mechanism

- All tests are CUDA C programs that interact with the tracking subsystem exclusively via the procfs interface; no modifications to application or kernel source are required per test
- Tests require root access and use `cudaMallocManaged` for UVM allocations
- The test suite is organised into three correctness categories:
  - **Table lifecycle:** init, destroy, and reinit behaviour; multi-region and stress scenarios
  - **Ordering:** write-before-start exclusion, timestamp monotonicity, non-destructive reads, first-write-wins under concurrent streams
  - **Address range filtering:** range isolation, subpage alignment, inverted and zero-length ranges, concurrent range-read races
- Performance is evaluated separately via five microbenchmarks covering write overhead, duplicate fault cost, contention, table lifecycle cost, and per-fault recording cost

Performance Analysis: Write Overhead

Measures kernel execution time and percentage overhead for a pure-write workload with tracking on vs. off, across varying allocation sizes.



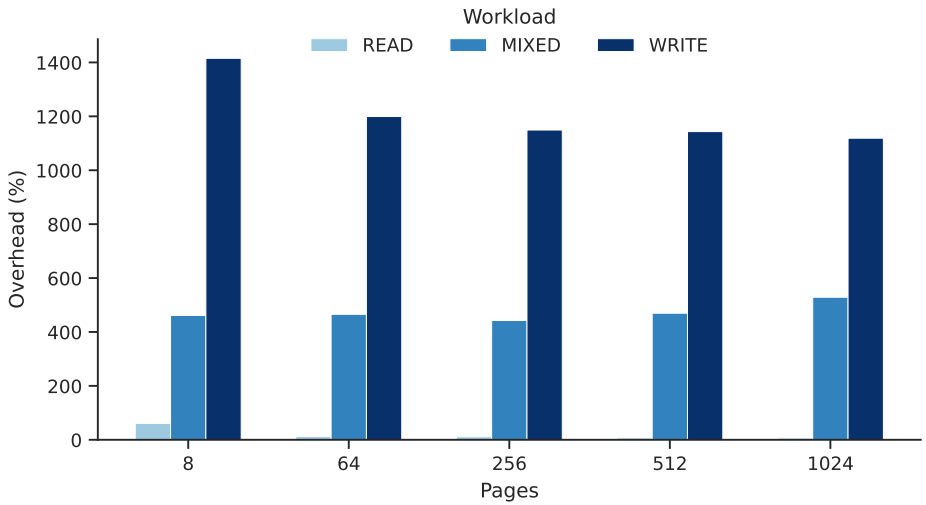


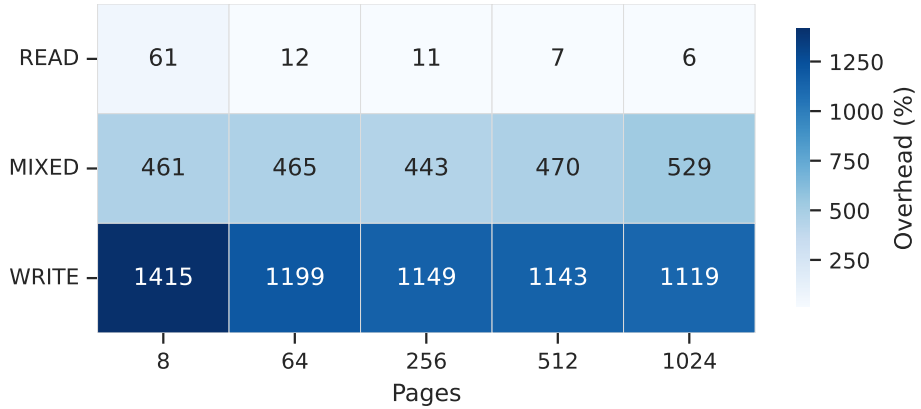
Both curves scale linearly; the tracked curve is ≈12× higher, reflecting per-page fault-servicing cost.

Overhead peaks at 1725% for small allocations where fixed invalidation cost dominates, converging to 1090–1160% at scale.

Performance Analysis: Workload Breakdown

Benchmarks READ, WRITE, and MIXED workloads across five page counts with tracking on and off, averaged over 10, 50, and 200 iterations.



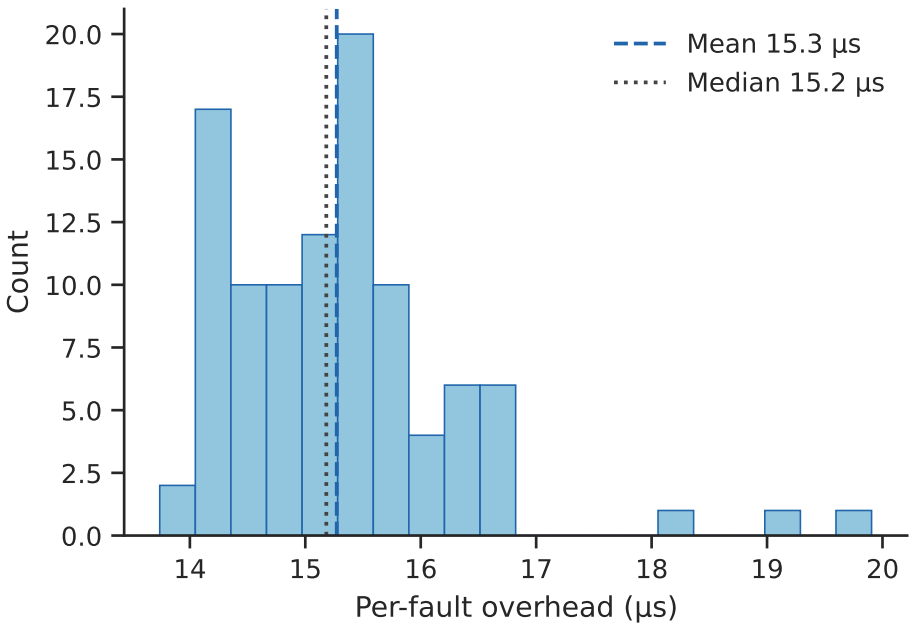


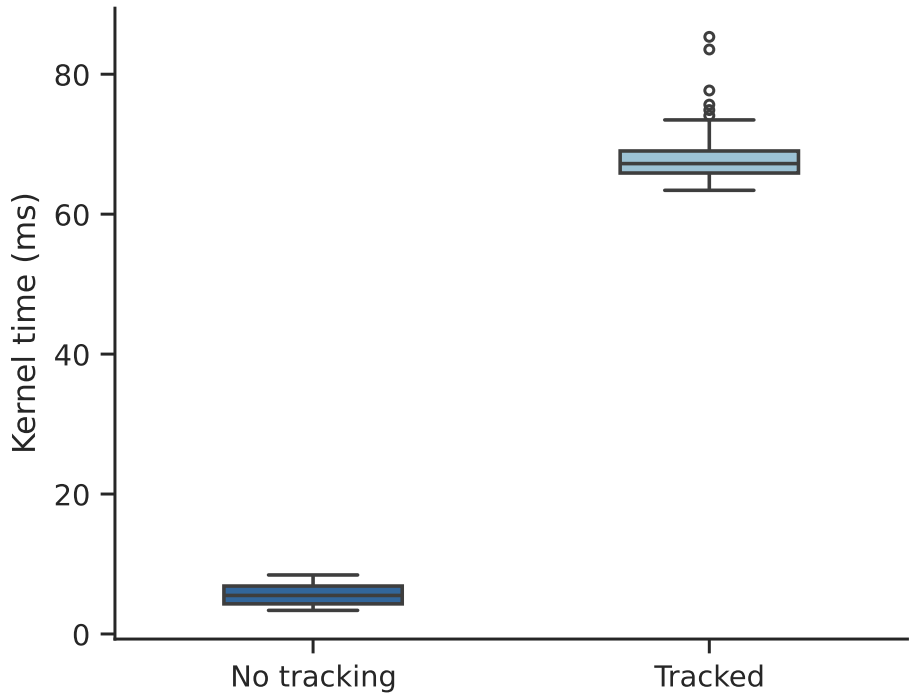
READ: 6–95%; MIXED: 415–497%; WRITE: 1090–1725%. Overhead is strictly proportional to write-faulting pages.

Heatmap confirms the trend; READ rows are uniformly cool, WRITE rows uniformly hot across all page counts.

Performance Analysis: Single Fault Recording Cost

Isolates the per-fault recording overhead using a single-thread kernel writing 4096 pages serially, run 100 times with and without tracking.



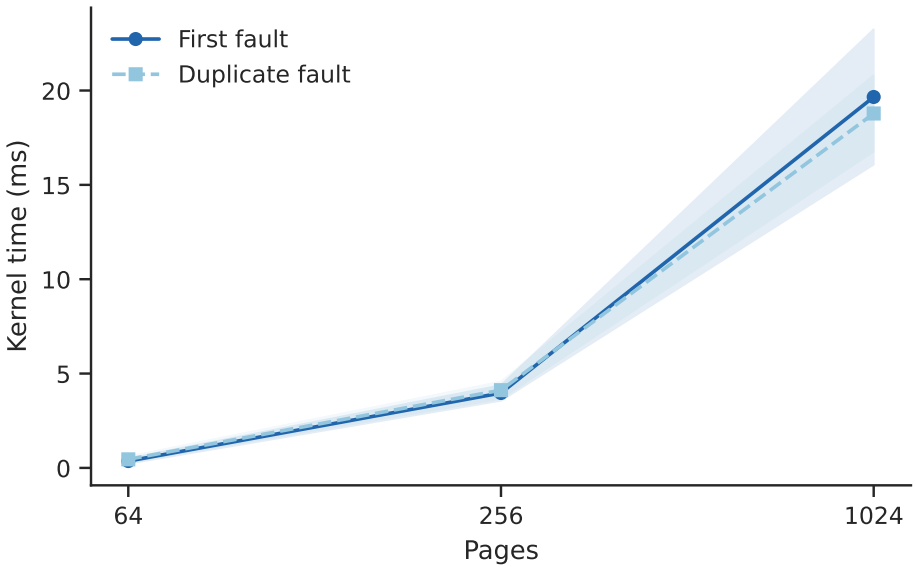


Per-fault overhead concentrated around $\approx$15.3 μs mean, indicating a stable and predictable per-page recording cost.

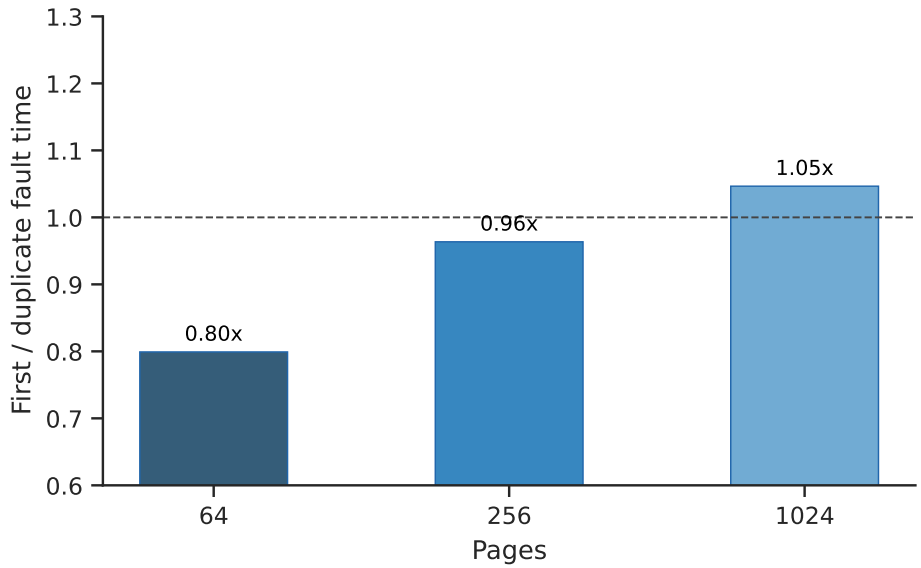
Boxplot confirms low variance across runs; the recording path behaves consistently with minimal outliers.

Performance Analysis: Duplicate Fault Skip Cost

Measures the cost of the first-write-wins skip path by writing the same pages twice within one epoch. Quantifies whether the xarray lookup on a duplicate fault is cheaper than a full insertion.



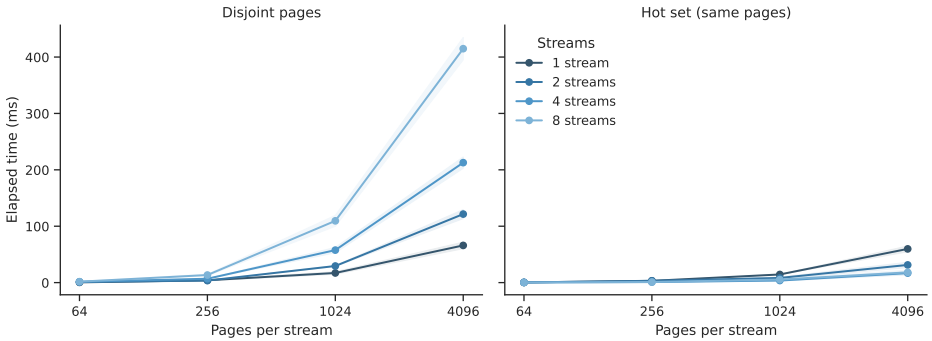
First and duplicate fault times track each other closely; the skip path offers little savings.

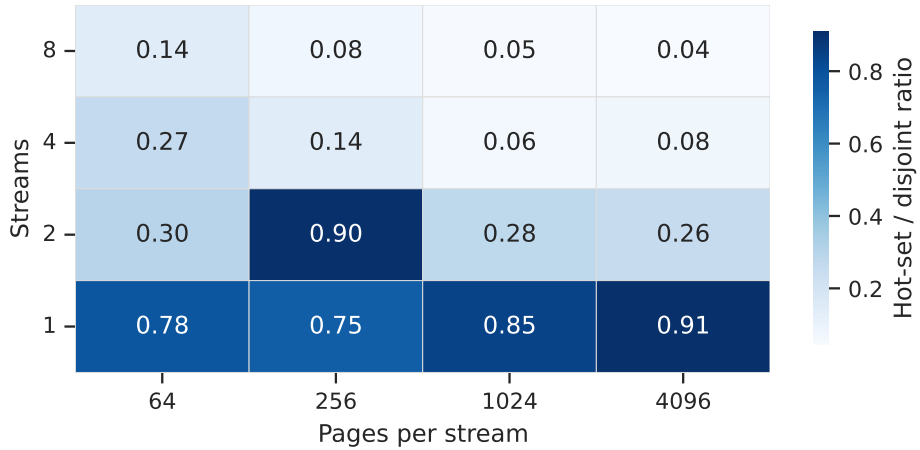


Ratio near 1.0 indicates the bottleneck is fault-servicing itself, not the xarray insertion.

Performance Analysis: Contention

Runs multiple CUDA streams concurrently under disjoint (each stream writes separate pages) and hot-set (all streams write the same pages) conditions.



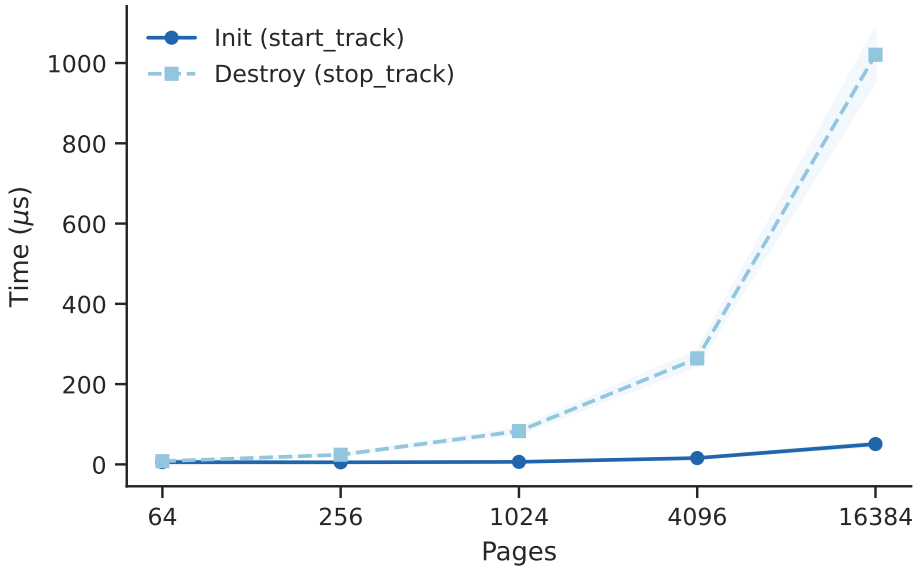


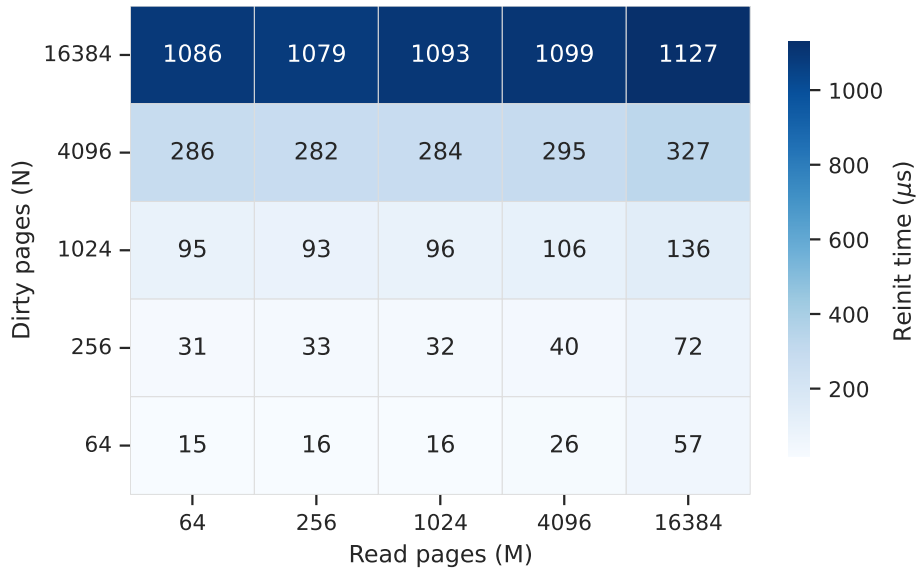
Hot-set elapsed times are consistently lower; concurrent streams hitting already-recorded pages trigger the skip path, reducing total work.

Hot-set advantage grows with stream count as skip-path hits increase.

Performance Analysis: Table Lifecycle Cost

Times the three procfs lifecycle operations: init (allocates empty xarray, invalidates GPU PTEs), destroy (frees all recorded entries), and reinit (destroy followed by re-invalidation).





Init is nearly flat at 5–51 μs ($O(1)$ xarray allocation); destroy scales linearly to $\approx$1 ms at 16384 pages.

Reinit cost rises steeply with dirty pages (N) and gently with read pages (M), as destroy dominates over PTE invalidation.

Ongoing Work: Profiling and Analysis

- **Function-specific profiling:** Insert `ktime` probes at entry and exit of individual UVM fault-handling functions to decompose total fault latency into per-function contributions
- **Deterministic profiling:** Current microbenchmarks show variance from GPU scheduling jitter; designing workloads with fixed, predictable access patterns to obtain stable measurements
- **Lock characterisation:** Instrumenting the range-filter spinlock and xarray internal locks to quantify contention under increasing stream counts and identify whether locks or fault-servicing dominate overhead
- **Memory footprint profiling:** Measuring per-entry xarray cost under large dirty sets; comparing against a bitmap representation to identify the crossover density at which a bitmap becomes more efficient

Next Steps: Optimisations

- **Write-permission removal:** Revoke write permissions only rather than full PTE invalidation at epoch start, reducing overhead for non-tracked and read-only pages
- **Parallel invalidation:** Distribute `va_block` invalidation across multiple kernel threads to make tracking onset more deterministic
- **Lockless recording:** Replace the coarse spinlock with per-entry locking or a fully lockless xarray path to reduce contention under high-concurrency workloads
- **Differential function priority:** Assign higher scheduling priority to fault-servicing functions on the critical path to reduce timing jitter
- **Bitmap representation:** For high-density dirty sets, replace the xarray with a bitmap to reduce memory overhead and improve read locality

Next Steps: Extensions and Deliverables

- **Tracking status query:** Lightweight procfs entry to check whether tracking is active for a given PID
- **CPU-side dirty tracking:** Extend to CPU writes via `mmu_notifier` integration, exposed through the existing procfs interface
- **Per-page write counters:** Track write frequency beyond first-write-wins to support migration and eviction decisions

Thank You